8. Részletes tervek

48 – GonoszOnosz

Konzulens:

Marcsingó Ádám

Csapattagok

Kálmán Patrik	XG5YQ1	kalmanpatrik1214@gmail.com
Papp Bálint	HND2AH	xypapbal@gmail.com
dr. Grób László Mihály	EAS7U0	laszlo.m.grob@gmail.com
Héja Gergő Zoltán	FEFSRB	hejagergo@gmail.com
Dobronyi Benedek Márk	IMQ1AR	benedek.dobronyi@gmail.com

2026.04.10

8. Részletes tervek

8.1 Osztályok és metódusok tervei.

1. ICommand

- **Felelősség:** Interfész, amely egy futtatható CLI parancsot reprezentál. Ezt az interfészt valósítja meg az összes konkrét bemeneti parancs (pl. Move_Bus, Load), biztosítva a parancsok egységes kezelését és a CommandParser kódjának módosítás nélküli bővíthetőségét.
- **Ősosztályok:** Nincs
- **Interfészek:** Nincs
- **Attribútumok:**
  - Nincs
- **Metódusok:**
  - + void execute(String[] args): Végrehajtja a parancsot a kapott string paramétertömb alapján.

2. CommandParser

- **Felelősség:** A bemeneti folyam (Standard Input vagy fájl) olvasása, a sorok darabolása és a megfelelő parancs (ICommand) dinamikus kikeresése és futtatása a regisztrációs szótár alapján. Hibás szintaxis esetén hibaüzenetet generál.
- **Ősosztályok:** Nincs
- **Interfészek:** Nincs
- **Attribútumok:**
  - - Map<String, ICommand> commandRegistry: Egy szótár, amely a parancsok szöveges neveit (pl. "move_bus", "create") köti össze a konkrét ICommand megvalósításokkal.
- **Metódusok:**
  - + void registerCommand(String name, ICommand command): Regisztrál egy új parancsot a rendszerbe.
    - **Algoritmus:**
      1. Eltárolja a commandRegistry szótárban a name paramétert mint kulcsot, és a command objektumot mint értéket.
  - + void parseStream(InputStream in): Egy végtelen ciklusban olvassa a bemeneti folyamot.
    - **Algoritmus:**
      1. Létrehoz egy olvasó (Scanner) objektumot az in bemenetre.
      2. AMÍG van következő sor a bemeneten: 2.1. Beolvassa a következő sort egy lokális line változóba. 2.2. Meghívja a saját parseLine(line) metódusát.
  - - void parseLine(String line): Egyetlen bemeneti sor értelmezése és a megfelelő parancs futtatása.
    - **Algoritmus:**
      1. Eltávolítja a line elejéről és végéről a felesleges szóközöket.
      2. HA a line üres, VAGY a line a "#" (komment) karakterrel kezdődik akkor visszatér (Kilép a metódusból).
      3. Darabolja fel a line változót a szóközök mentén, az eredményt mentse el az args string tömbbe.
      4. Legyen a commandName az args tömb legelső eleme (args[0]).
      5. HA a commandRegistry szótár tartalmazza a commandName kulcsot, akkor. Kikéri a szótárból a commandName-hez tartozó

ICommand objektumot. Meghívja az objektum execute(args) metódusát.
6. KÜLÖNBEN hibát jelez.

ObjectManager

- **Felelősség:** A rendszerben lévő objektumok (mezők, járművek, játékosok) szöveges azonosító (ID) alapú nyilvántartása. A CLI parancsok stringeket használnak, a belső játéklogika referenciákat; ez az osztály teremti meg a kapcsolatot közöttük.
- **Ősosztályok:** Nincs
- **Interfészek:** Nincs
- **Attribútumok:**
  - - Map<String, Object> objects: Egy általános szótár, amely az összes regisztrált objektumot tárolja az ID-ja alapján.
- **Metódusok:**
  - + void registerObject(String id, Object obj): Új objektum regisztrálása a rendszerbe (pl. create vagy spawn parancsoknál).
    - **Algoritmus:**
      1. HA az objects szótár már tartalmazza az id kulcsot, AKKOR Meghívja az hibajelző
      2. Eltárolja az objects szótárban az id-t és az obj-t.
  - + Object getObject(String id): Visszaad egy objektumot az azonosítója alapján.
    - **Algoritmus:**
      1. HA az objects szótár NEM tartalmazza az id kulcsot, AKKOR meghívja az hibajelző metódust.
      2. VISSZATÉRÉSI ÉRTÉK a szótárban lévő objektum.
  - + void clearAll(): Törli a teljes memóriaképet (a load parancs használja betöltés előtt).
    - **Algoritmus:**
      1. Kiüríti az objects szótárat.

OutputFormatter

- **Felelősség:** Központosított segédosztály a konzolos kimenetek formázására. Biztosítja, hogy a prototípus kimenete tökéletesen megfeleljen a specifikáció által elvárt [SUCCESS], [ERROR], [EVENT] és [STATE] prefixumoknak.
- **Ősosztályok:** Nincs
- **Interfészek:** Nincs
- **Attribútumok:** Nincs (Statikus osztály)
- **Metódusok:**
  - + static void printSuccess(String message): Sikeres akció kiírása.
    - **Algoritmus:**
      1. Kiírja a Standard Outputra a "[SUCCESS] " szöveget és hozzáfűzi a message paramétert.
  - + static void printError(String message): Hibaüzenet kiírása.
    - **Algoritmus:**
      1. Kiírja a Standard Outputra az "[ERROR] " szöveget és hozzáfűzi a message paramétert.
  - + static void printEvent(String message): Rendszeresemény kiírása.
    - **Algoritmus:**
      1. Kiírja a Standard Outputra az "[EVENT] " szöveget és hozzáfűzi a message paramétert.

StateLoader

- **Felelősség:** A load parancs kiszolgálása. Megnyit egy konfigurációs szövegfájlt, és annak tartalmát soronként beolvasva átadja a CommandParser-nek végrehajtásra, mintha a felhasználó gépelte volna be azokat.
- **Ősosztályok:** Nincs
- **Interfészek:** Nincs
- **Attribútumok:**
  - - CommandParser parser: Referencia a rendszer parancsértelmezőjére.
  - - ObjectManager objectManager: Referencia az objektum nyilvántartóra.
- **Metódusok:**
  - + void loadFromFile(String filename): Pálya betöltése fájlból.
    - **Algoritmus:**
      1. az eddigi állapot törlése.
      2. Kísérletet tesz a filename nevű fájl megnyitására olvasásra.
      3. HA a fájl megnyitása sikertelen (nem található), error jelzése
      4. Létrehoz egy bemeneti folyamot a fájlhoz.
      5. Meghívja a parser.parseStream() metódust a fájl bemeneti folyamával.
      6. Siker jelzése

StateSaver

- **Felelősség:** A save parancs kiszolgálása. Lekérdezi az ObjectManager-ből az aktuális memóriaképet (az összes mezőt, járművet, kapcsolatot), és visszagenerálja belőlük azokat a bemeneti parancsokat (pl. create, spawn), amikkel a jelenlegi állapot újra felépíthető.
- **Ősosztályok:** Nincs
- **Interfészek:** Nincs
- **Attribútumok:**
  - - ObjectManager objectManager: Referencia az objektum nyilvántartóra az adatok kinyeréséhez.
- **Metódusok:**
  - + void saveToFile(String filename): A memóriakép kimentése.
    - **Algoritmus:**
      1. Kísérletet tesz a filename nevű fájl megnyitására írásra. (Ha a filename üres, a Standard Outputra fog írni).
      2. Lekéri az objectManager-ből az összes tárolt objektumot.
      3. CIKLUS MINDEN objektumra a gyűjteményből: 3.1. HA az objektum típusa Field, AKKOR kigenerálja a create lane/building/stb. parancsot az ID-val. 3.2. HA az objektum típusa Vehicle, AKKOR kigenerálja a spawn ... parancsot a megfelelő paraméterekkel. 3.3. Kiírja a generált parancsot a fájlba (vagy a kimenetre).
      4. CIKLUS MINDEN Field objektumra: 4.1. Lekérdezi a szomszédait (getNeighbors()). 4.2. Kigenerálja a connect_fields parancsokat és kiírja őket.
      5. Lezárja a fájlt.
      6. HA fájlba írtunk, AKKOR siker logolása

ProtoApp

- **Felelősség:** A prototípus alkalmazás fő belépési pontja (Main osztály). Összeszereli a keretrendszert: példányosítja a CommandParsert, az ObjectManagert, a GameLogic-ot, beregisztrálja a parancsokat, és elindítja a Standard Input (konzol) olvasását.
- **Ősosztályok:** Nincs

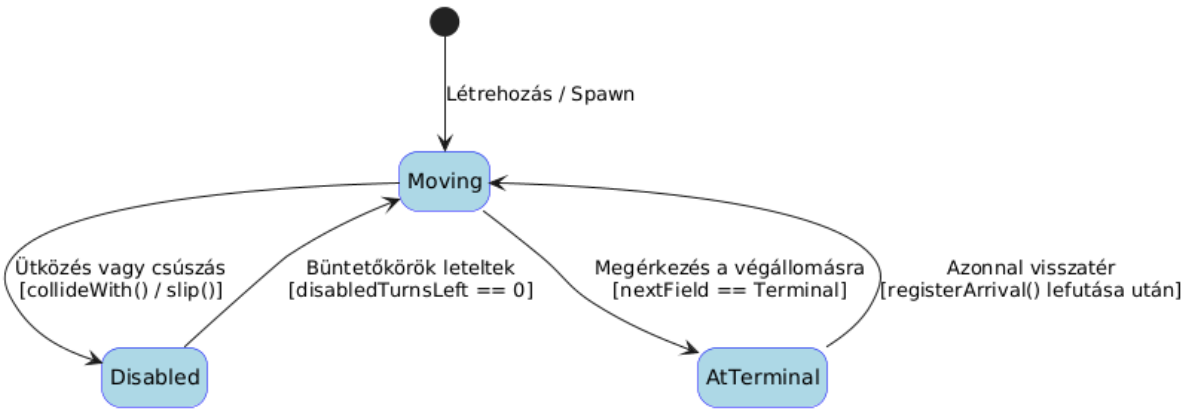
- **Interfészek:** Nincs
- **Attribútumok:** Nincs
- **Metódusok:**
  - + static void main(String[] args): A program belépési pontja.
    - **Algoritmus:**
      1. Példányosít egy új ObjectManager objektumot.
      2. Példányosít egy új GameLogic objektumot (átadva neki a szükséges függőségeket).
      3. Példányosít egy új CommandParser objektumot.
      4. Beregisztrálja az összes parancsot a parserbe (parser.registerCommand("move_bus", new MoveBus (registry, gameLogic)), stb.).
      5. Meghívja a parser.parseStream(Standard Input) metódust, amely átveszi a vezérlést.

Building

- **Felelősség:** Az összes épülettípus (GeneralBuilding, HomeBase, Terminal) közös absztrakt ősosztálya. Implementálja a Field interfészt. Felelőssége, hogy egységes alapot biztosítson az épületeknek a szomszédossági kapcsolatok kezeléséhez.
- **Ősosztályok:** Nincs
- **Interfészek:** Field
- **Attribútumok:**
  - + List<Field> neighbors: A szomszédos mezők listája.
  - - List<Vehicle> parkedVehicles: Privát lista az épületben aktuálisan tartózkodó/parkoló járművek tárolására.
- **Metódusok:**
  - + void accept(Vehicle v): Fogadja az érkező járművet.
    - **Algoritmus:**
      1. A v jármű hozzáadása a parkedVehicles listához.
  - + void remove(Vehicle v): Eltávolítja a távozó járművet.
    - **Algoritmus:**
      1. A v jármű eltávolítása a parkedVehicles listából.
  - + List<Field> getNeighbors(): Visszaadja a szomszédos mezőket.
    - **Algoritmus:**
      1. VISSZATÉRÉSI ÉRTÉK a neighbors attribútum értéke.

Bus

- **Felelősség:** A buszvezető játékos által irányított jármű, amely két végállomás között közlekedik. Fordulókat teljesít; ütközés vagy csúszás esetén meghatározott ideig mozgásképtelenné válik és elzárja a sávot. Állapotok és átmenetek: A busz három állapotban lehet. Mozgás (Moving): normálisan közlekedik a buszvezető irányításával. Mozgásképtelen (Disabled): ütközés vagy csúszás után kerül ide; a disabledTurnsLeft csökkentésével tér vissza Mozgásba. Végállomáson (AtTerminal): megérkezett a végállomásra, a Terminal.registerArrival() hívódik, majd azonnal visszamegy Mozgásba.



-
- **Ősosztályok:** Vehicle -> Bus
- **Interfészek:** Nincs
- **Attribútumok:**
  - + bool isFunctioning: Hamis értéke esetén a busz ütközés miatt mozgásképtelen.
  - + int disabledTurnsLeft: Hány körig marad még mozgásképtelen.
  - - String name: A busz vizuális és logikai azonosítására szolgáló név.
  - - int PENALTY_TURNS: Konstans érték, amely megadja, hogy pontosan hány kört kell kihagyni egy ütközés vagy megcsúszás bekövetkezésekor.
- **Metódusok:**
  - + void move(): Mozgatja a buszt a tervezett következő mező felé.
    - **Algoritmus:**
      1. HA isFunctioning hamis AKKOR 1.1. disabledTurnsLeft csökkentése 1-gyel. 1.2. HA disabledTurnsLeft egyenlő 0 AKKOR 1.2.1. isFunctioning beállítása igazra.
      2. HA nextField nem üres AKKOR 2.1. A currentField elem remove(this) metódusának hívása. 2.2. A nextField elem accept(this) metódusának hívása. 2.3. HA nextField típusa Terminal AKKOR 2.3.1. A nextField elem registerArrival(this) metódusának hívása. 2.4. currentField beállítása nextField értékére. 2.5. nextField beállítása üresre.
  - + void collideWith(Vehicle v): Ütközést kezel egy másik járművel, beállítja a mozgásképtelen állapotot.
    - **Algoritmus:**
      1. setPenalty(PENALTY_TURNS) hívása.
  - + void slip(): Jégpáncélon történő megcsúszást kezel.
    - **Algoritmus:**
      1. setPenalty(PENALTY_TURNS) hívása.
      2. CIKLUS MINDEN v elemre a currentField.vehicles listában 2.1. HA v nem egyenlő önmagával AKKOR 2.1.1. A v.collideWith(this) metódus hívása.
      3. A currentField elem setBlocked(PENALTY_TURNS) metódusának hívása.
  - + String getName(): Visszaadja a busz nevét.
    - **Algoritmus:**
      1. VISSZATÉRÉSI ÉRTÉK a name attribútum értéke.
  - - void setPenalty(int turns): Privát segédmetódus a büntetési körök beállítására.
    - **Algoritmus:**
      1. isFunctioning beállítása hamisra.

2. disabledTurnsLeft beállítása a turns értékre.

Car

- **Felelősség:** A városlakók automatizált járműve, amely önállóan navigál lakása és munkahelye között a legrövidebb járható útvonalon. Ha vékony hórétegen halad, letapossa a havat; jégen csúszhat és balesetet okozhat; mély hóban elakad, de szomszédos szabad sávba átsorolhat. Állapotok és átmenetek: Moving (Mozgásban): normálisan halad az útvonalán. Disabled (Mozgásképtelen): ütközés vagy csúszás után kerül ide (waitTurns = PENALTY_TURNS); minden körben a waitTurns csökken, 0-nál visszatér Moving-ba.
- **Ősosztályok:** Vehicle -> Car
- **Interfészek:** Nincs
- **Attribútumok:**
  - + Building homeBuilding: A lakás épülete, ahonnan indul.
  - + Building workBuilding: A munkahely épülete, ahova tart.
  - + Map gameMap: A térkép referenciája az útvonal-számításhoz.
  - - boolean isGoingToWork: Privát logikai változó, amely tárolja, hogy az autó éppen a munkahelyére tart-e (igaz) vagy hazafelé (hamis).
  - - int PENALTY_TURNS: Privát konstans a baleset utáni várakozási körök számára.
  - - int waitTurns: Privát számláló, ami tárolja, hány körből marad még ki az autó baleset miatt.
- **Metódusok:**
  - + void move(): A kocsi a kiszámolt útvonal alapján lép a következő mezőre.
    - **Algoritmus:**
      1. HA waitTurns nagyobb mint 0 AKKOR 1.1. waitTurns csökkentése 1-gyel. 1.2. VISSZATÉRÉSI ÉRTÉK
      2. calculatePath() hívása.
      3. HA nextField nem üres AKKOR 3.1. A currentField elem remove(this) metódusának hívása. 3.2. A nextField elem accept(this) metódusának hívása. 3.3. HA nextField típusa Lane AKKOR 3.3.1. A nextField elem applyCompaction() metódusának hívása. 3.4. currentField beállítása nextField értékére. 3.5. nextField beállítása üresre.
      4. HA currentField megegyezik a célépülettel AKKOR 4.1. isGoingToWork értékének megfordítása (ellentettjére állítása).
  - + void calculatePath(): Meghívja a térkép útvonalkeresőjét és beállítja a következő mezőt.
    - **Algoritmus:**
      1. HA isGoingToWork igaz AKKOR célpont = workBuilding, KÜLÖNBEN célpont = homeBuilding.
      2. Útvonal lista = a gameMap.findShortestPath(currentField, célpont) hívása.
      3. HA Útvonal lista nem üres AKKOR 3.1. nextField beállítása az Útvonal lista első elemére.
      4. KÜLÖNBEN 4.1. changeLane() hívása.
  - + void collideWith(Vehicle v): Ütközés kezelése, sávlezárás meghatározott körre.
    - **Algoritmus:**
      1. waitTurns beállítása a PENALTY_TURNS értékére.

2. HA currentField típusa Lane AKKOR 2.1. A currentField elem setBlocked(PENALTY_TURNS) metódusának hívása.
- + void slip(): Jégpáncélon megcsúszás kezelése.
  - **Algoritmus:**
    1. waitTurns beállítása a PENALTY_TURNS értékére.
    2. CIKLUS MINDEN v elemre a jelenlegi mező járműlistájából 2.1. HA v nem egyenlő önmagával AKKOR 2.1.1. A v.collideWith(this) metódus hívása.
    3. HA currentField típusa Lane AKKOR 3.1. A currentField elem setBlocked(PENALTY_TURNS) metódusának hívása.
- + void changeLane(): Átsorolás a szomszédos szabad sávba elakadás esetén.
  - **Algoritmus:**
    1. Szomszédok lista = a currentField.getNeighbors() metódus hívása.
    2. CIKLUS MINDEN szomszed elemre a Szomszédok listából 2.1. HA szomszed típusa Lane ÉS a szomszed isBlocked() metódusa hamis AKKOR 2.1.1. nextField beállítása a szomszed értékére. 2.1.2. VISSZATÉRÉSI ÉRTÉK

BusDriver

- **Felelősség:** A buszt vezető játékos, akinek célja, hogy a busz a végállomások (Terminal) között ingázzon. A játékos irányítja a busz mozgását, és minden teljesített fordulóért (amikor a busz eléri a végállomást) pontot kap, amit a rendszer regisztrál.
- **Ősosztályok:** Player -> BusDriver
- **Interfészek:** Nincs
- **Attribútumok:**
  - - int completedRounds: Privát számláló, a játékos által eddig teljesített sikeres fordulók száma.
- **Metódusok:**
  - + void startTurn(): Elkezdi a buszvezető körét, várja a parancsot.
    - **Algoritmus:**
      1. Rendszerüzenet generálása, hogy a játékos köre következik.
      2. Bemeneti parancs (pl. move_bus) bekérése és átadása a CommandParser-nek.
  - + void endTurn(): Befejezi a buszvezető körét.
    - **Algoritmus:**
      1. Átadja a vezérlést a következő játékosnak vagy a GameLogic-nak.
  - + void incrementRounds(): Növeli a fordulók számát és a játékos pontszámát.
    - **Algoritmus:**
      1. A completedRounds értékének növelése 1-gyel.

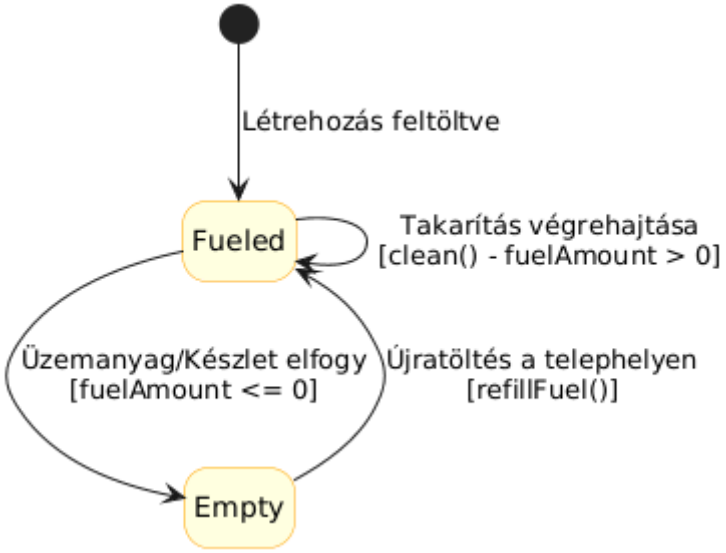
Cleaner

- **Felelősség:** A hókotrót irányító játékos. Célja a sávok takarítása, amiért pénzjutalmat kap, amit az ősosztály money attribútuma tárol. A megszerzett pénzből a telephelyen (HomeBase) a beépített boltból (IntegratedMarket) új tisztítófejeket és üzemanyagot vásárolhat.
- **Ősosztályok:** Player -> Cleaner
- **Interfészek:** Nincs
- **Attribútumok:**
  - - List<SnowPlow> controlledPlows: A játékos irányítása alatt álló hókotrók.

- **Metódusok:**
  - + void startTurn(): Elkezdi a takarító körét, várja a parancsot.
    - **Algoritmus:**
      1. Rendszerüzenet generálása, hogy a takarító köre következik (pénzösszeg kijelzésével).
      2. Bemeneti parancs (pl. move_plow, buy, equip) bekérése és átadása a CommandParser-nek.
  - + void endTurn(): Befejezi a takarító körét.
    - **Algoritmus:**
      1. Átadja a vezérlést a következő játékosnak vagy a GameLogic-nak.
  - + void addMoney(int amount): Pénz hozzáadása sikeres takarítás után.
    - **Algoritmus:**
      1. A money értékének növelése az amount értékével.
  - + void addPlow(SnowPlow sp): Hozzáad egy gépet a flottához.
    - **Algoritmus:**
      1. Az sp hókotrót hozzáadja a controlledPlows listához.
      2. Meghívja az sp.setOwner(this) metódust az összerendelés véglegesítéséhez.

CleanerHead

- **Felelősség:** A hókotróra szerelhető cserélhető eszközök absztrakt ősosztálya. Meghatározza, hogy a hókotró hogyan hat a sávon lévő hóra és jégre.
  A töltetet igénylő fejek state chartja(a helytakarékosság miatt csak itt feltüntetve)



- **Ősosztályok:** Nincs
- **Interfészek:** IPurchasable
- **Attribútumok:**
  - + int price: A fej ára a boltban.
  - + bool usesFuel: Használ-e üzemanyagot.
  - - String name: A hókotró fej privát megnevezése.
- **Metódusok:**
  - + void clean(Lane lane): A konkrét fejre jellemző takarítási műveletet elvégzi.
    - **Algoritmus:** Absztrakt metódus, nincs konkrét implementációja.
  - + int getPrice(): Visszaadja a fej árát.

        - **Algoritmus:**
            1. VISSZATÉRÉSI ÉRTÉK a price attribútum értéke.
    - + String getName(): Visszaadja a fej nevét.
        - **Algoritmus:**
            1. VISSZATÉRÉSI ÉRTÉK a name attribútum értéke.

DragonHead

- **Felelősség:** A legdrágább és leghatékonyabb hókotró fej. Gázturbinával azonnal elolvasztja a sávon lévő teljes havat és jégpáncélt. Működéséhez biokerozin szükséges. Állapotok: Fueled (Üzemanyaggal): a fej aktívan működik, Empty (Üres): nincs biokerozin, a clean nem hat.
- **Ősosztályok:** CleanerHead -> DragonHead
- **Interfészek:** Nincs
- **Attribútumok:**
    - + double fuelAmount: A jelenleg tárolt biokerozin mennyisége.
    - + double fuelPrice: A tárolt üzemanyag ára.
    - - double FUEL_CONSUMPTION_RATE: Privát konstans, amely megadja az egy sáv takarításához szükséges üzemanyag mennyiséget.
- **Metódusok:**
    - + void clean(Lane lane): Azonnal eltávolítja az összes havat és jeget a sávról.
        - **Algoritmus:**
            1. HA fuelAmount nagyobb mint 0 AKKOR 1.1. A lane.removeSnow(lane.snowDepth) metódus hívása. 1.2. A lane.removeIce() metódus hívása. 1.3. fuelAmount csökkentése a FUEL_CONSUMPTION_RATE értékével. 1.4. HA fuelAmount kisebb mint 0 AKKOR fuelAmount beállítása 0-ra.
    - + void refillFuel(double amount): Biokerozint tölt a tartályba a telephelyen.
        - **Algoritmus:**
            1. fuelAmount növelése az amount értékével.

Field

- **Felelősség:** A gráf alapköve: minden pályaelem ezt az interfészt valósítja meg. Biztosítja, hogy a járművek egységesen tudjanak mozogni a gráfban.
- **Ősosztályok:** Nincs
- **Interfészek:** Nincs
- **Attribútumok:** Nincs
- **Metódusok:**
    - + void accept(Vehicle v): Fogadja az érkező járművet.
        - **Algoritmus:** Interfész, nincs implementáció.
    - + void remove(Vehicle v): Eltávolítja a távozó járművet.
        - **Algoritmus:** Interfész, nincs implementáció.
    - + List<Field> getNeighbors(): Visszaadja a szomszédos mezők listáját.
        - **Algoritmus:** Interfész, nincs implementáció.

GameLogic

- **Felelősség:** A játék menetét vezérlő osztály. Felelőssége: a körök szervezése, az autók navigálásának indítása, a játék végének detektálása és az eredmény összesítése.
- **Ősosztályok:** Nincs
- **Interfészek:** Nincs
- **Attribútumok:**

- + Map gameMap: A játék térképe.
- + List<Player> players: A játékban részt vevő játékosok listája.
- + int turnCount: Az aktuális kör sorszáma.
- + int maxTurns: A játék hossza körökben.
- + List<Vehicle> vehicles: Az összes jármű listája.
- - boolean isGameOver: Privát logikai változó, amely tárolja, hogy véget ért-e a játék.

- **Metódusok:**
  - + void startGame(): Inicializálja és elindítja a játékot.
    - **Algoritmus:**
      1. A gameMap attribútum inicializálása és feltöltése mezőkkel.
      2. A players és vehicles listák inicializálása.
      3. turnCount beállítása 0-ra.
      4. isGameOver beállítása hamisra.
  - + void advanceTurn(): Eggyel növeli a körszámlálót, végrehajtja a végrehajtási fázist.
    - **Algoritmus:**
      1. HA isGameOver igaz AKKOR VISSZATÉRÉSI ÉRTÉK.
      2. turnCount növelése 1-gyel.
      3. HA turnCount nagyobb vagy egyenlő mint maxTurns AKKOR 3.1. endGame() hívása. 3.2. VISSZATÉRÉSI ÉRTÉK
      4. A gameMap.snow() metódus hívása.
      5. CIKLUS MINDEN v elemre a vehicles listában 5.1. A v.move() metódus hívása.
      6. A manageTurn() metódus hívása.
  - + void manageTurn(): Kezeli az aktív játékosok döntési fázisát.
    - **Algoritmus:**
      1. CIKLUS MINDEN p elemre a players listában 1.1. A p.startTurn() metódus hívása. 1.2. Játékos döntéseinek bevárása és feldolgozása. 1.3. A p.endTurn() metódus hívása.
  - + void endGame(): Befejezi a játékot és kiszámítja az eredményt.
    - **Algoritmus:**
      1. isGameOver beállítása igazra.
      2. A legmagasabb pontszám és legtöbb pénz kiszámítása a players lista alapján.
      3. A győztes(ek) adatainak megjelenítése.

GeneralBuilding

- **Felelősség:** Az autók által felkeresett általános épület (lakás vagy munkahely). Felelőssége parkolóhelyet biztosítani az autóknak és navigációs célpontként szolgálni.
- **Ősosztályok:** Building -> GeneralBuilding
- **Interfészek:** Nincs
- **Attribútumok:** *(A Building ősosztály attribútumait használja)*
- **Metódusok:**
  - + void accept(Vehicle v): Fogadja az érkező autót.
    - **Algoritmus:**
      1. Az ősosztály accept(v) metódusának hívása.
  - + void remove(Vehicle v): Eltávolítja a távozó autót.
    - **Algoritmus:**
      1. Az ősosztály remove(v) metódusának hívása.

- + List<Field> getNeighbors(): Visszaadja a szomszédos mezőket.
  - **Algoritmus:**
    1. VISSZATÉRÉSI ÉRTÉK a neighbors attribútum értéke.

GravelHead

- **Felelősség:** Zúzalékot (zúzott követ) szóró hókotró fej. A kiszórt zúzalék megszünteti a jég csúszósságát, de a hó befedheti. Működéséhez zúzalék (mint üzemanyag) szükséges ; ha a készlet kiürül, a takarítás nem hat. Állapotok és átmenetek: *Fueled (Üzemanyaggal)*: a fej aktívan működik és szórja a zúzalékot. *Empty (Üres)*: a zúzalék elfogyott, a fej nem csinál semmit. Újratöltés után visszakerül *Fueled* állapotba.
- **Ősosztályok:** CleanerHead -> GravelHead
- **Interfészek:** Nincs
- **Attribútumok:**
  - + double fuelAmount: A jelenleg tárolt zúzalék mennyisége.
  - + double fuelPrice: A tárolt üzemanyag (zúzalék) ára.
  - - double GRAVEL_CONSUMPTION_RATE: Privát konstans, az egy sávra kiszórt zúzalék mennyisége.
- **Metódusok:**
  - + void clean(Lane lane): Zúzalékot juttat a sávra, megakadályozva a csúszásveszélyt, és felhasználva a zúzalékból a szükséges mennyiséget.
    - **Algoritmus:**
      1. HA a fuelAmount nagyobb mint 0 AKKOR 1.1. A lane.addGravel(GRAVEL_CONSUMPTION_RATE) metódus hívása. 1.2. A fuelAmount csökkentése a GRAVEL_CONSUMPTION_RATE értékével. 1.3. HA a fuelAmount kisebb mint 0 AKKOR a fuelAmount beállítása 0-ra.
  - + void refillFuel(double amount): Zúzalékot tölt a tartályba a telephelyen.
    - **Algoritmus:**
      1. A fuelAmount növelése az amount értékével.

HomeBase

- **Felelősség:** A hókotrók bázisa. Felelőssége a hókotrók fogadása, a fejcserék és az üzemanyag-utántöltések koordinálása. A beépített áruháznak is helyet ad.
- **Ősosztályok:** Building -> HomeBase
- **Interfészek:** Nincs
- **Attribútumok:**
  - + IntegratedMarket market: A telephelyen lévő bolt.
  - + int capacity: A telephely maximális befogadóképessége.
  - - List<SnowPlow> dockedPlows: Privát lista a telephelyen aktuálisan tartózkodó hókotrók nyilvárntartására.
- **Metódusok:**
  - + void acceptSnowPlow(SnowPlow sp): Fogadja a visszatérő hókotrót, belső kapacitáskezeléssel.
    - **Algoritmus:**
      1. HA a dockedPlows elemeinek száma kisebb, mint a capacity AKKOR 1.1. Az sp hozzáadása a dockedPlows listához. 1.2. Az ősosztály accept(sp) metódusának hívása.

- + void refuelSnowPlow(SnowPlow sp, double amount): Utántölti a megadott hókotró üzemanyag-tartályát.
  - **Algoritmus:**
    1. HA az sp.activeHead típusa SaltHead vagy DragonHead AKKOR 1.1. Az sp.activeHead.refillFuel(amount) metódus hívása.
- + void swapHead(SnowPlow sp, CleanerHead newHead): Fejcserét hajt végre a megadott hókotróra.
  - **Algoritmus:**
    1. HA a dockedPlows lista tartalmazza az sp elemet AKKOR 1.1. Az sp.changeHead(newHead) metódus hívása.
- + void accept(Vehicle v): Fogadja az érkező járművet.
  - **Algoritmus:**
    1. Az ősosztály accept(v) metódusának hívása.
- + void remove(Vehicle v): Eltávolítja a távozó járművet.
  - **Algoritmus:**
    1. Az ősosztály remove(v) metódusának hívása.
- + List<Field> getNeighbors(): Visszaadja a szomszédos mezőket.
  - **Algoritmus:**
    1. VISSZATÉRÉSI ÉRTÉK a neighbors attribútum értéke.

IcebreakerHead

- **Felelősség:** Speciális hókotró fej, amely a jégpáncél feltörésére specializálódott. Havat nem takarít el, üzemanyagot nem igényel.
- **Ősosztályok:** CleanerHead -> IcebreakerHead
- **Interfészek:** Nincs
- **Attribútumok:** *(A CleanerHead attribútumait örökli)*
- **Metódusok:**
  - + void clean(Lane lane): A sávon lévő jégpáncélt feltöri és havává alakítja.
    - **Algoritmus:**
      1. HA a lane.isFrozen attribútum értéke igaz AKKOR 1.1. A lane.breakIce() metódus hívása.

IntegratedMarket

- **Felelősség:** A telephelyen lévő bolt, amelyből a takarítók eszközöket és üzemanyagot vásárolhatnak. Felelőssége a vásárlási tranzakciók lebonyolítása.
- **Ősosztályok:** Nincs
- **Interfészek:** Nincs
- **Attribútumok:**
  - + List<IPurchasable> availableItems: A boltban kapható termékek listája.
- **Metódusok:**
  - + boolean buyItem(Player buyer, IPurchasable item): Megvásárol egy terméket.
    - **Algoritmus:**
      1. HA a buyer.money nagyobb vagy egyenlő, mint az item.getPrice() értéke AKKOR 1.1. A buyer.money csökkentése az item.getPrice() értékével. 1.2. VISSZATÉRÉSI ÉRTÉK igaz.
      2. KÜLÖNBEN 2.1. VISSZATÉRÉSI ÉRTÉK hamis.
  - + List<IPurchasable> getAvailableItems(): Visszaadja a megvásárolható termékek listáját.

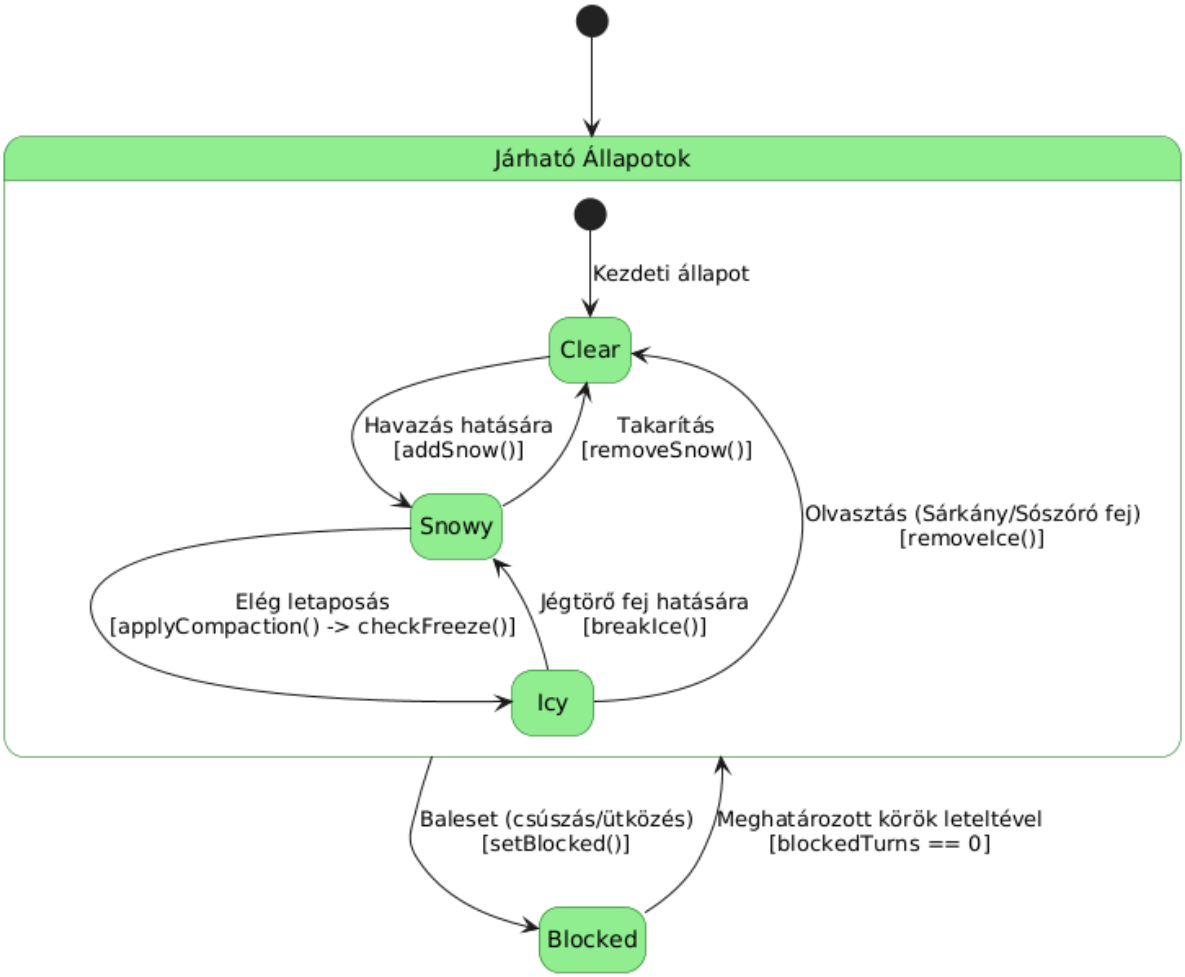
- **Algoritmus:**
  1. VISSZATÉRÉSI ÉRTÉK az availableItems lista.

9. IPurchasable

- **Felelősség:** Közös interfész minden olyan objektum számára, amely a játékbeli boltban (IntegratedMarket) megvásárolható (például CleanerHead leszármazottak, SnowPlow). Biztosítja, hogy a bolt egységesen le tudja kérdezni az árucikkek árát és nevét a tranzakciók lebonyolításához.
- **Ősosztályok:** Nincs
- **Interfészek:** Nincs
- **Attribútumok:** Nincs
- **Metódusok:**
  - + int getPrice(): Visszaadja az adott termék megvásárlásához szükséges összeget.
    - **Algoritmus:** (Absztrakt metódus, az implementációt a konkrét megvásárolható osztályok adják meg).
  - + String getName(): Visszaadja a termék nevét vagy típusát, amelyet a bolt a listázáskor és a nyugtázáskor használ.
    - **Algoritmus:** (Absztrakt metódus, az implementációt a konkrét megvásárolható osztályok adják meg).

Lane

- **Felelősség:** Az úthálózat gráfjának alapcsomópontja: egy forgalmi sáv, amelyen a járművek közlekednek. Felelőssége nyilvántartani a hóréteg vastagságát, a jégpáncél állapotát és a tartózkodó járműveket. Állapotok és átmenetek: Tiszta (Clear): nincs hó/jég. Havazás hatására átmegy Havas állapotba. Havas (Snowy): hóréteg van rajta. Elég letaposásra Jeges állapotba megy, takarítással visszamegy Tiszta állapotba. Jeges (Icy): jégpáncél van rajta. Jégtörő hatására visszamegy Havas állapotba. Lezárt (Blocked): baleset miatt; meghatározott körök után visszanyílik.



- **Ősosztályok:** Nincs
- **Interfészek:** Field
- **Attribútumok:**
  - + double snowDepth: A sávon lévő hóréteg vastagsága.
  - + bool isFrozen: Igaz, ha a sávon jégpáncél van.
  - + int compactionCount: Hányszor hajtottak át járművek a sávon.
  - + List<Vehicle> vehicles: A sávon tartózkodó járművek listája.
  - + double saltEffect: A sóhatás fennmaradásának ideje.
  - + List<Field> neighbors: A szomszédos mezők listája.
  - - int COMPACTION_THRESHOLD: Konstans, a letaposási küszöbérték a fagyáshoz.
  - - double ICE_BREAK_SNOW_AMOUNT: Privát konstans, a jégtörés után visszamaradó hó mennyisége.
  - + double gravelDepth: A sávon lévő zúzalékréteg vastagsága.
  - - int blockedTurns: A sávlezárásból hátralévő körök száma.
- **Metódusok:**
  - + void accept(Vehicle v): Fogadja az érkező járművet.
    - **Algoritmus:**
      1. A v hozzáadása a vehicles listához.
      2. HA az isFrozen attribútum igaz AKKOR 2.1. Csúszás esélyének legenerálása. 2.2. HA a generált esély alapján a jármű megcsúszik AKKOR 2.2.1. A v.slip() metódus hívása.
  - + void remove(Vehicle v): Eltávolítja a távozó járművet.

        - **Algoritmus:**
            1. A v eltávolítása a vehicles listából.
    - + List<Field> getNeighbors(): Visszaadja a szomszédos mezőket.
        - **Algoritmus:**
            1. VISSZATÉRÉSI ÉRTÉK a neighbors attribútum értéke.
    - + void addSnow(double amount): Növeli a hóréteg vastagságát.
        - **Algoritmus:**
            1. HA a saltEffect nagyobb mint 0 AKKOR 1.1. VISSZATÉRÉSI ÉRTÉK
            2. A snowDepth növelése az amount értékével.
    - + void removeSnow(double amount): Csökkenti a hóréteg vastagságát.
        - **Algoritmus:**
            1. A snowDepth csökkentése az amount értékével.
            2. HA a snowDepth kisebb vagy egyenlő mint 0 AKKOR 2.1. A snowDepth beállítása 0-ra.
            3. A compactionCount beállítása 0-ra.
    - + void breakIce(): A jégpáncélt feltöri és havává alakítja.
        - **Algoritmus:**
            1. HA az isFrozen igaz AKKOR 1.1. Az isFrozen beállítása hamisra. 1.2. Az addSnow(ICE_BREAK_SNOW_AMOUNT) metódus hívása.
    - + void removeIce(): Eltávolítja a jégpáncélt.
        - **Algoritmus:**
            1. Az isFrozen beállítása hamisra.
    - + void checkFreeze(): Ellenőrzi, hogy kell-e jégpáncél képződjön.
        - **Algoritmus:**
            1. HA a compactionCount nagyobb vagy egyenlő, mint a COMPACTION_THRESHOLD ÉS a snowDepth nagyobb mint 0 AKKOR 1.1. Az isFrozen beállítása igazra. 1.2. A snowDepth beállítása 0-ra. 1.3. A compactionCount beállítása 0-ra.
    - + void applyCompaction(): Növeli a letaposás számlálót.
        - **Algoritmus:**
            1. HA a snowDepth nagyobb mint 0 AKKOR 1.1. A compactionCount növelése 1-gyel. 1.2. A checkFreeze() metódus hívása.
    - + void applySaltEffect(double duration): Beállítja a sóhatást a sávon.
        - **Algoritmus:**
            1. A saltEffect beállítása a duration értékére.
            2. A removeSnow(snowDepth) metódus hívása.
            3. A removeIce() metódus hívása.
    - + boolean isBlocked(): Igaz, ha a sáv járhatatlan.
        - **Algoritmus:**
            1. HA a blockedTurns nagyobb mint 0 AKKOR 1.1. VISSZATÉRÉSI ÉRTÉK igaz.
            2. KÜLÖNBEN 2.1. VISSZATÉRÉSI ÉRTÉK hamis.
    - - void setBlocked(int turns): Privát segédmetódus a sáv lezárására.
        - **Algoritmus:**
            1. A blockedTurns beállítása a turns értékre.
    - - void updateTurnEffects(): Privát segédmetódus a körönkénti lezárások és sóhatás frissítésére.

        - **Algoritmus:**
            1. HA a saltEffect nagyobb mint 0 AKKOR 1.1. A saltEffect csökkentése 1-gyel.
            2. HA a blockedTurns nagyobb mint 0 AKKOR 2.1. A blockedTurns csökkentése 1-gyel.
    - + void addGravel(double amount): Növeli a sávon lévő zúzalék mennyiségét.
        - **Algoritmus:**
            1. A gravelDepth növelése az amount értékével.
    - + void removeGravel(double amount): Csökkenti a sávon lévő zúzalék mennyiségét.
        - **Algoritmus:**
            1. A gravelDepth csökkentése az amount értékével.
            2. HA a gravelDepth kisebb vagy egyenlő mint 0 AKKOR 2.1. A gravelDepth beállítása 0-ra.

Map

- **Felelősség:** Összefogja és kezeli az úthálózat összes csomópontját. Biztosít útvonal-kereső funkcionalitást és koordinálja a havazást.
- **Ősosztályok:** Nincs
- **Interfészek:** Nincs
- **Attribútumok:**
    - + List<Field> fields: A térkép összes mezője.
    - - double SNOW_AMOUNT_PER_TURN: Privát konstans a körönként hulló hó vastagságához.
- **Metódusok:**
    - + List<Field> findShortestPath(Field from, Field to): Visszaadja a legrövidebb járható útvonalat.
        - **Algoritmus:**
            1. Útvonal lista inicializálása.
            2. Szélességi bejárás (BFS) vagy Dijkstra algoritmus futtatása a fields listán a from csomóponttól a to csomópontig.
            3. HA a cél csomópont elérhető AKKOR 3.1. A bejárt útvonal hozzáadása az Útvonal listához.
            4. VISSZATÉRÉSI ÉRTÉK az Útvonal lista.
    - + void snow(): Minden Lane típusú mezőn növeli a hóréteg vastagságát.
        - **Algoritmus:**
            1. CIKLUS MINDEN f elemre a fields listában 1.1. HA az f típusa Lane AKKOR 1.1.1. Az ((Lane)f).addSnow(SNOW_AMOUNT_PER_TURN) metódus hívása.

Player

- **Felelősség:** A játékos absztrakt osztály, a Cleaner és a BusDriver közös ősosztálya. Tárolja az alapvető tulajdonságokat.
- **Ősosztályok:** Nincs
- **Interfészek:** Nincs
- **Attribútumok:**
    - + String name: A játékos neve.
    - + int score: A játékos aktuális pontszáma.

    - + double money: A játékos aktuális pénze.
- **Metódusok:**
    - + void startTurn(): Elkezdi az adott játékos körét.
        - **Algoritmus:** Absztrakt metódus.
    - + void endTurn(): Befejezi az adott játékos körét.
        - **Algoritmus:** Absztrakt metódus.

Road

- **Felelősség:** Az úthálózat egy útját reprezentálja. Összefogja a sávokat és épületeket.
- **Ősosztályok:** Nincs
- **Interfészek:** Nincs
- **Attribútumok:**
    - + List<Road> startPoint: A szomszédos utak listája.
    - + List<Road> endPoint: A szomszédos utak listája.
    - + List<Lane> forwardLanes: A startból endbe haladó sávok.
    - + List<Lane> backwardLanes: Az endből startba haladó sávok.
    - + List<Building> buildings: Az utcához csatlakozó épületek.
    - + Double length: Az út hossza.
- **Metódusok:**
    - + List<Road> getStartConnections(): Visszaadja a kezdőpont kapcsolatait.
        - **Algoritmus:**
            1. VISSZATÉRÉSI ÉRTÉK a startPoint attribútum.
    - + List<Road> getEndConnections(): Visszaadja a végpont kapcsolatait.
        - **Algoritmus:**
            1. VISSZATÉRÉSI ÉRTÉK az endPoint attribútum.
    - + List<Lane> getForwardLanes(): Visszaadja az előre sávokat.
        - **Algoritmus:**
            1. VISSZATÉRÉSI ÉRTÉK a forwardLanes attribútum.
    - + List<Lane> getBackwardLanes(): Visszaadja a vissza sávokat.
        - **Algoritmus:**
            1. VISSZATÉRÉSI ÉRTÉK a backwardLanes attribútum.
    - + void addLane(Lane lane, String direction): Hozzáad egy sávot a megfelelő irányú sávlistához.
        - **Algoritmus:**
            1. HA a direction értéke "forward" AKKOR 1.1. A lane hozzáadása a forwardLanes listához.
            2. KÜLÖNBEN 2.1. A lane hozzáadása a backwardLanes listához.
    - + void addBuilding(Building b): Hozzáad egy épületet az utcához.
        - **Algoritmus:**
            1. A b hozzáadása a buildings listához.
    - + void setLength(double length): Beállítja az út hosszát.
        - **Algoritmus:**
            1. A length attribútum beállítása a paraméterre.
    - + void connectTo(Road other, String myPos, String otherPos): Két út logikai összekötése a kezdő- vagy végponton.
        - **Algoritmus:**
            1. HA myPos értéke "start" AKKOR 1.1. Az other hozzáadása a startPoint listához.
            2. KÜLÖNBEN 2.1. Az other hozzáadása az endPoint listához.

SaltHead

- **Felelősség:** Sót szóró hókotró fej. A kiszórt só idővel felolvasztja a havat és jégpáncélt. Állapotok: Fueled (Üzemanyaggal): aktív, Empty (Üres): nem szór sót.
- **Ősosztályok:** CleanerHead -> SaltHead
- **Interfészek:** Nincs
- **Attribútumok:**
  - + double fuelAmount: A jelenleg tárolt só mennyisége.
  - + double fuelPrice: A tárolt só ára.
  - - int SALT_DURATION_TURNS: Privát konstans, a sóhatás időtartama körökben.
  - - double SALT_CONSUMPTION_RATE: Privát konstans, egy sávra jutó só mennyisége.
- **Metódusok:**
  - + void clean(Lane lane): Sót szór a sávra, beállítja a sóhatás időtartamát.
    - **Algoritmus:**
      1. HA a fuelAmount nagyobb mint 0 AKKOR 1.1. A lane.applySaltEffect(SALT_DURATION_TURNS) metódus hívása. 1.2. A fuelAmount csökkentése a SALT_CONSUMPTION_RATE értékével. 1.3. HA a fuelAmount kisebb mint 0 AKKOR a fuelAmount beállítása 0-ra.
  - + void refillFuel(double amount): Sót tölt a tartályba a telephelyen.
    - **Algoritmus:**
      1. A fuelAmount növelése az amount értékével.

SnowPlow

- **Felelősség:** A takarító játékos által irányított munkagép. A rászerelt fej segítségével takarítja a sávokat, pénzt termel. Hóláncai miatt jégen sem csúszik. Állapotok és átmenetek: AtHomeBase (Telephelyen): a telephelyen áll, changeHead és üzemanyag-utántöltés engedélyezett. OnRoad (Úton): úton halad, ezek a műveletek [ERROR]-t adnak. Az átmenetet a move() utáni currentField típusa határozza meg: ha HomeBase, akkor AtHomeBase, egyébként OnRoad.
- **Ősosztályok:** Vehicle -> SnowPlow
- **Interfészek:** IPurchasable
- **Attribútumok:**
  - + List<CleanerHead> attachments: A jelenleg birtokolt hókotró fejek listája.
  - + CleanerHead activeHead: A jelenleg felszerelt hókotró fej.
  - - int price: Privát változó a jármű boltban látható megvásárlási árához.
  - - String name: Privát változó a hókotró azonosító nevéhez.
  - - Cleaner owner: A jármű tulajdonosa, aki a döntéseket hozza és a jutalmat kapja.
- **Metódusok:**
  - + void move(): Mozgatja a hókotrót a tervezett útvonal szerint.
    - **Algoritmus:**
      1. HA a nextField nem üres AKKOR 1.1. A currentField.remove(this) metódus hívása. 1.2. A nextField.accept(this) metódus hívása. 1.3. A currentField beállítása a nextField értékére. 1.4. A cleanCurrentLane() metódus hívása. 1.5. A nextField beállítása üresre.

- + void changeHead(CleanerHead newHead): Kicseréli az aktív fejet a megadottra.
  - **Algoritmus:**
    1. HA az attachments lista tartalmazza a newHead elemet AKKOR 1.1. Az activeHead beállítása a newHead értékére.
- + void cleanCurrentLane(): Végrehajtja az aktív fejre jellemző takarítást a jelenlegi sávon.
  - **Algoritmus:**
    1. HA az activeHead nem üres ÉS a currentField típusa Lane AKKOR 1.1. Az activeHead.clean((Lane)currentField) metódus hívása.
- + int getPrice(): Visszaadja az árat.
  - **Algoritmus:**
    1. VISSZATÉRÉSI ÉRTÉK a price attribútum értéke.
- + String getName(): Visszaadja a nevet.
  - **Algoritmus:**
    1. VISSZATÉRÉSI ÉRTÉK a name attribútum értéke.
- + void setOwner(Cleaner c): Beállítja a tulajdonost.
  - **Algoritmus:**
    1. Az owner attribútum értékét beállítja a paraméterként kapott c takarítóra.

SweepHead

- **Felelősség:** Söprő típusú fej. A havat a szomszédos sávba vagy az út mellé tolja. Jeget nem tud eltávolítani.
- **Ősosztályok:** CleanerHead -> SweepHead
- **Interfészek:** Nincs
- **Attribútumok:** *(A CleanerHead attribútumait örökli)*
- **Metódusok:**
  - + void clean(Lane lane): A sáv havát áttol a szomszédos sávba.
    - **Algoritmus:**
      1. Az sAmount lokális változó beállítása a lane.snowDepth értékére.
      2. A gAmount lokális változó beállítása a lane.gravelDepth értékére.
      3. A lane.removeSnow(sAmount) metódus hívása.
      4. A lane.removeGravel(gAmount) metódus hívása.
      5. Szomszédok lista = a lane.getNeighbors() metódus hívása.
      6. CIKLUS MINDEN s elemre a Szomszédok listából 4.1. HA az s típusa Lane AKKOR 4.1.1. Az ((Lane)s).addSnow(sAmount / Szomszédok lista hossza) metódus hívása. 4.1.2. Az ((Lane)s).addGravel(gAmount / Szomszédok lista hossza) metódus hívása.

Terminal

- **Felelősség:** A buszjáratok végpontja. Felelőssége a fordulók regisztrálása: amikor egy busz megérkezik, jelzi a buszvezetőnek a forduló teljesítését.
- **Ősosztályok:** Building -> Terminal
- **Interfészek:** Nincs
- **Attribútumok:** *(A Building attribútumait örökli)*
- **Metódusok:**

- + void registerArrival(Bus bus): Regisztrálja a busz érkezését és jelzi a fordulót.
  - **Algoritmus:**
    1. A Játékos objektum (BusDriver) kikeresése a memóriából.
    2. A Játékos score értékének növelése 1-gyel.
- + void accept(Vehicle v): Fogadja az érkező buszt.
  - **Algoritmus:**
    1. Az ősosztály accept(v) metódusának hívása.
- + void remove(Vehicle v): Eltávolítja a távozó buszt.
  - **Algoritmus:**
    1. Az ősosztály remove(v) metódusának hívása.
- + List<Field> getNeighbors(): Visszaadja a szomszédos mezőket.
  - **Algoritmus:**
    1. VISSZATÉRÉSI ÉRTÉK a neighbors attribútum értéke.

ThrowerHead

- **Felelősség:** A havat jóval messzebbre szórja, nem csupán a szomszédos sávba (így az útról eltűnik). Jeget nem tud eltávolítani.
- **Ősosztályok:** CleanerHead -> ThrowerHead
- **Interfészek:** Nincs
- **Attribútumok:**
  - + double throwDistance: A távolság, ameddig a hányó fej a havat elhajítja.
- **Metódusok:**
  - + void clean(Lane lane): A sáv havát messzire szórja el az útról.
    - **Algoritmus:**
      1. Az sAmount lokális változó beállítása a lane.snowDepth értékére.
      2. A gAmount lokális változó beállítása a lane.gravelDepth értékére.
      3. A lane.removeSnow(sAmount) metódus hívása.
      4. A lane.removeGravel(gAmount) metódus hívása.
      5.

Vehicle

- **Felelősség:** A játékban szereplő összes jármű absztrakt ősosztálya. Tárolja a jármű aktuális sávját és a tervezett következő mezőt.
- **Ősosztályok:** Nincs
- **Interfészek:** Nincs
- **Attribútumok:**
  - + Field currentField: A jármű aktuális mezője (sáv vagy épület).
  - + Field nextField: A tervezett következő mező.
- **Metódusok:**
  - + void move(): A jármű mozgatása a jelenlegi helyzetről a következő tervezett mezőre.
    - **Algoritmus:** Absztrakt metódus, implementációja a konkrét leszármazott osztályokban található.

9.1 A tesztek részletes tervei, leírásuk a teszt nyelvén

A bemeneti nyelv kiegészítései a 7. heti leíráshoz képest:

- set_salt_effect <lane_id> <duration>: Tesztelési célra közvetlenül beállítja a sáv saltEffect attribútumát a megadott körszámra. Példa: set_salt_effect l1 2.0
- spawn snowplow <id> <field_id> <owner_id>: A hókotró spawnolása most kötelezően 4 paramétert vár. Az owner_id egy már létező cleaner játékos azonosítója, akinek a hókotró a tulajdonába kerül.
- set_lane_state <lane_id> <snowDepth> <isFrozen> <iceThickness> <gravelDepth>: Az iceThickness paraméter a parancs szignatúrájában maradt kompatibilitási okokból (0.0 értékkel kitölthető), de a belső modellben már nincs iceThickness attribútum — a parancs beolvasásakor az érték egyszerűen eldobódik.

MoveBusCommand (minta az ICommand implementációjához)

- **Felelősség:** A move_bus <bus_id> <target_field_id> parancs végrehajtása. Kikeresi az ID-k alapján a Bus és Field objektumokat, ellenőrzi a szomszédsági kapcsolatot, majd meghívja a busz move() metódusát. A többi 19 konkrét parancs ezzel azonos sémán épül.
- **Ősosztályok:** Nincs
- **Interfészek:** ICommand
- **Attribútumok:**
  - - ObjectManager objectManager: Referencia az objektum nyilvántartóhoz az ID-alapú feloldáshoz.
- **Metódusok:**
  - + void execute(String[] args): Végrehajtja a parancsot a kapott string paramétertömb alapján.
    - **Algoritmus:**
      3. HA args.length kisebb mint 3 AKKOR hibajelzés ("invalid syntax") és visszatérés.
      4. bus = objectManager.getObject(args[1]) mint Bus.
      5. target = objectManager.getObject(args[2]) mint Field.
      6. HA bus vagy target üres AKKOR hibajelzés ("object not found") és visszatérés.
      7. HA target nem szerepel a bus.currentField.getNeighbors() listában AKKOR hibajelzés ("not a neighbor") és visszatérés.
      8. A bus.nextField beállítása a target értékére.
      9. A bus.move() metódus hívása.
      10. Siker jelzése: OutputFormatter.printSuccess("<bus_id> moved to <target_id>").

Teszteset 1: Pályabetöltés és teljes memóriakép

- **Leírás:** Egy egyszerű konfigurációs fájlt töltünk be: egy utat két sávval, egy HomeBase-t, egy Terminalt és egy hókotrót. A betöltés után a list és stat parancsokkal lekérdezzük az összes elemet. Az elvárt kimenetben pontosan a beállított értékeknek kell szerepelniük, beleértve a snowDepth=0, isFrozen=false kezdeti értékeket.
- **Ellenőrzött funkcionalitás, várható hibahelyek:** Map osztály inicializálása, CommandParser create/spawn/connect_fields parancsai, Field-ek ID-alapú nyilvántartása, stat kimenet konzisztenciája.
- **Bemenet:**

create road r1
create lane l1
create lane l2
add_to_road r1 lane l1 forward
add_to_road r1 lane l2 forward

connect_fields l1 l2
set_road_length r1 100.0
create homebase hb1
create terminal t1
connect_fields l2 t1
connect_fields l1 hb1
create cleaner c1
spawn snowplow sp1 hb1 c1
list lanes
list vehicles
stat l1
stat sp1

- **Elvárt kimenet:**

[SUCCESS] road r1 created
[SUCCESS] lane l1 created
[SUCCESS] lane l2 created
[SUCCESS] lane l1 added to road r1 as forward
[SUCCESS] lane l2 added to road r1 as forward
[SUCCESS] l1 connected to l2
[SUCCESS] road r1 length set to 100.0
[SUCCESS] homebase hb1 created
[SUCCESS] terminal t1 created
[SUCCESS] l2 connected to t1
[SUCCESS] l1 connected to hb1
[SUCCESS] cleaner c1 created
[SUCCESS] sp1 spawned at hb1
[LIST] lanes: l1, l2
[LIST] vehicles: sp1
[STATE] l1 (Lane)
  snowDepth: 0.0
  isFrozen: false
  gravelDepth: 0.0
  saltEffect: 0.0
  isBlocked: false
  vehicles: []
[STATE] sp1 (SnowPlow)
  currentField: hb1
  activeHead: SweepHead
  fuelAmount: 0.0
  attachments: []

Teszteset 2: Mentés és visszatöltés (Save/Load)

- **Leírás:** Betöltünk egy pályát, módosítunk rajta (hó hozzáadása tesztelési céllal set_lane_state-tel), majd kimentjük a save paranccsal. Ezután újraindítjuk a programot, betöltjük a mentett fájlt, és ellenőrizzük, hogy az állapot megegyezik-e. Ez teszteli a bemeneti és kimeneti nyelv kompatibilitását.
- **Ellenőrzött funkcionalitás, várható hibahelyek:** A save és load parancs, valamint a kimeneti nyelv 100%-os kompatibilitása a bemeneti nyelvvel.
- **Bemenet:**

create lane l1
set_lane_state l1 15.0 false 0.0 5.0

save

- **Elvárt kimenet:**

[SUCCESS] lane l1 created
[SUCCESS] l1 state set
create lane l1
set_lane_state l1 15.0 false 0.0 5.0

Teszteset 3: Busz végállomásra érése

- **Leírás:** Betöltünk egy egyszerű pályát egy busszal (l1-en) és egy terminállal (t1, l1 szomszédja). A buszt a terminálra léptetjük move_bus paranccsal. Az elvárt kimenet tartalmazza a [SUCCESS] és [EVENT] üzeneteket, a buszvezető completedRounds száma 1-re nő.
- **Ellenőrzött funkcionalitás, várható hibahelyek:** Bus.move(), Terminal.registerArrival(), BusDriver.incrementRounds(), a pontszámítás.
- **Bemenet:**

create lane l1
create terminal t1
connect_fields l1 t1
create busdriver bd1
spawn bus bus1 l1
move_bus bus1 t1
stat bd1

- **Elvárt kimenet:**

[SUCCESS] lane l1 created
[SUCCESS] terminal t1 created
[SUCCESS] l1 connected to t1
[SUCCESS] busdriver bd1 created
[SUCCESS] bus1 spawned at l1
[SUCCESS] bus1 moved to t1
[EVENT] bus1 completed a round at t1
[STATE] bd1 (BusDriver)
  name: bd1
  score: 0
  completedRounds: 1

Teszteset 4: Érvénytelen mozgás nem szomszédos mezőre

- **Leírás:** A hókotrót egy olyan sávra próbáljuk léptetni, amellyel nincs közvetlen kapcsolata a gráfban. A rendszernek [ERROR] üzenetet kell adnia, a hókotró pozíciója nem változhat.
- **Ellenőrzött funkcionalitás, várható hibahelyek:** Field.getNeighbors() ellenőrzése, érvénytelen mozgás kiszűrése a move() metódusban.
- **Bemenet:**

create lane l1
create lane l2
create lane l3
connect_fields l1 l2
create homebase hb1
connect_fields l1 hb1
create cleaner c1
spawn snowplow sp1 hb1 c1
move_plow sp1 l1
move_plow sp1 l3

stat sp1

- **Elvárt kimenet:**

[SUCCESS] lane l1 created
[SUCCESS] lane l2 created
[SUCCESS] lane l3 created
[SUCCESS] l1 connected to l2
[SUCCESS] homebase hb1 created
[SUCCESS] l1 connected to hb1
[SUCCESS] cleaner c1 created
[SUCCESS] sp1 spawned at hb1
[SUCCESS] sp1 moved to l1
[ERROR] l3 is not a neighbor of l1
[STATE] sp1 (SnowPlow)
  currentField: l1
  activeHead: SweepHead
  fuelAmount: 0.0
  attachments: []

Teszteset 5: NPC autó útvonaltervezése

- **Leírás:** Létrehozunk egy autót és kiadjuk a next_turn parancsot, szimulálva az idő múlását. Az autó pozíciójának lekérdezésekor látszódnia kell, hogy lépett a célja felé.
- **Ellenőrzött funkcionalitás, várható hibahelyek:** A Car.calculatePath() algoritmus és az autonóm move() lépések helyességének vizsgálata a körökre osztott rendszerben.
- **Bemenet:**

random off
create lane l_1
create lane l_2
connect_fields l_1 l_2
create building b_home
create building b_work
connect_fields b_home l_1
connect_fields b_work l_2
create car car_1
spawn car car_1 b_home b_home b_work
next_turn
stat car_1

- **Elvárt kimenet:**

[SUCCESS] random mode set to off
[SUCCESS] lane l_1 created
[SUCCESS] lane l_2 created
[SUCCESS] fields l_1 and l_2 connected
[SUCCESS] building b_home created
[SUCCESS] building b_work created
[SUCCESS] fields b_home and l_1 connected
[SUCCESS] fields b_work and l_2 connected
[SUCCESS] car car_1 created
[SUCCESS] car_1 spawned at b_home
[EVENT] car_1 moved to l_1
[STATE] car_1 (Car)
  currentField: l_1
  homeBuilding: b_home

workBuilding: b_work
isGoingToWork: true
waitTurns: 0

Teszteset 6: NPC autó reakciója lezárt sávra

- **Leírás:** Egy haladó autó előtti sávot (ahová lépne) blokkolt állapotba hozzuk egy másik, jégen megcsúszott járművel. A next_turn után ellenőrizzük, hogy az autó elkerüli-e a lezárt sávot.
- **Ellenőrzött funkcionalitás, várható hibahelyek:** A járművek akadálykeresési logikájának és a Lane.isBlocked() attribútum visszatartó erejének tesztelése.
- **Bemenet:**

```
random off
create lane l_1
create lane l_2
connect_fields l_1 l_2
set_lane_state l_2 0.0 true 5.0 0.0
create building b_home
create building b_work
connect_fields b_home l_1
connect_fields b_work l_2
create car dummy_car
spawn car dummy_car l_2 b_home b_work
force_slip dummy_car true
create car car_1
spawn car car_1 l_1 b_home b_work
next_turn
stat car_1
```

- **Elvárt kimenet:**

```
[SUCCESS] random mode set to off
[SUCCESS] lane l_1 created
[SUCCESS] lane l_2 created
[SUCCESS] fields l_1 and l_2 connected
[SUCCESS] lane l_2 state updated
[SUCCESS] building b_home created
[SUCCESS] building b_work created
[SUCCESS] fields b_home and l_1 connected
[SUCCESS] fields b_work and l_2 connected
[SUCCESS] car dummy_car created
[SUCCESS] dummy_car spawned at l_2
[SUCCESS] force_slip set to true for dummy_car
[SUCCESS] car car_1 created
[SUCCESS] car_1 spawned at l_1
[EVENT] dummy_car slipped on l_2. Field is now blocked.
[STATE] car_1 (Car)
  currentField: l_1
  homeBuilding: b_home
  workBuilding: b_work
  isGoingToWork: true
  waitTurns: 0
```

Teszteset 7: Determinisztikus megcsúszás jégen

- **Leírás:** Bekapcsoljuk a determinisztikus módot. Létrehozunk egy jeges sávot, majd egy buszt léptetünk rá. Mivel random off módban és force_slip true be van állítva, garantáltan megcsúszik. A sávnak lezártnak kell lennie.
- **Ellenőrzött funkcionalitás, várható hibahelyek:** Vehicle.slip(), Lane.setBlocked(), determinisztikus mód (random off), force_slip parancs.
- **Bemenet:**

```
create lane l1
create lane l2
connect_fields l1 l2
set_lane_state l2 0.0 true 5.0 0.0
create busdriver bd1
spawn bus bus1 l1
random off
force_slip bus1 true
move_bus bus1 l2
stat l2
```

- **Elvárt kimenet:**

```
[SUCCESS] lane l1 created
[SUCCESS] lane l2 created
[SUCCESS] l1 connected to l2
[SUCCESS] l2 state set
[SUCCESS] busdriver bd1 created
[SUCCESS] bus1 spawned at l1
[SUCCESS] random mode set to off
[SUCCESS] force_slip bus1 set to true
[SUCCESS] bus1 moved to l2
[EVENT] bus1 slipped on l2. Field is now blocked.
[STATE] l2 (Lane)
  snowDepth: 0.0
  isFrozen: true
  gravelDepth: 0.0
  saltEffect: 0.0
  isBlocked: true
  vehicles: []
```

Teszteset 8: Megcsúszás kikényszerítése és elkerülése (force_slip)

- **Leírás:** Egy jeges sávra lépés előtt kiadjuk a force_slip <id> false parancsot a járműre. A lépés után a sávnak járhatónak kell maradnia.
- **Ellenőrzött funkcionalitás, várható hibahelyek:** A teszteléshez készített force_slip parancs működésének ellenőrzése.
- **Bemenet:**

```
random off
create lane l_1
create lane l_2
connect_fields l_1 l_2
set_lane_state l_2 0.0 true 5.0 0.0
create building b1
create building b2
create car car_1
spawn car car_1 l_1 b1 b2
force_slip car_1 false
```

next_turn
stat l_2

- **Elvárt kimenet:**

[SUCCESS] random mode set to off
[SUCCESS] lane l_1 created
[SUCCESS] lane l_2 created
[SUCCESS] fields l_1 and l_2 connected
[SUCCESS] lane l_2 state updated
[SUCCESS] building b1 created
[SUCCESS] building b2 created
[SUCCESS] car car_1 created
[SUCCESS] car_1 spawned at l_1
[SUCCESS] force_slip set to false for car_1
[EVENT] car_1 moved to l_2
[STATE] l_2 (Lane)
  snowDepth: 0.0
  isFrozen: true
  gravelDepth: 0.0
  saltEffect: 0.0
  isBlocked: false
  vehicles: []

Teszteset 9: Hókotró takarítása söprő fejjel (hó áthelyezése)

- **Leírás:** Egy sávra 20 cm havat állítunk be. A hókotrót söprő fejjel rávezetjük. Az elvárt kimenet: a takarított sávon snowDepth = 0, a szomszéd sávon snowDepth megnő.
- **Ellenőrzött funkcionalitás, várható hibahelyek:** SweepHead.clean(), Lane.removeSnow(), Lane.addSnow(), szomszédsági kapcsolat kezelése.
- **Bemenet:**

create lane l1
create lane l2
create lane l3
connect_fields l1 l2
connect_fields l2 l3
set_lane_state l2 20.0 false 0.0 0.0
create homebase hb1
connect_fields l1 hb1
create cleaner c1
spawn snowplow sp1 hb1 c1
move_plow sp1 l1
move_plow sp1 l2
stat l2
stat l1

- **Elvárt kimenet:**

[SUCCESS] lane l1 created
[SUCCESS] lane l2 created
[SUCCESS] lane l3 created
[SUCCESS] l1 connected to l2
[SUCCESS] l2 connected to l3
[SUCCESS] l2 state set
[SUCCESS] homebase hb1 created

[SUCCESS] l1 connected to hb1
[SUCCESS] cleaner c1 created
[SUCCESS] sp1 spawned at hb1
[SUCCESS] sp1 moved to l1
[SUCCESS] sp1 moved to l2
[STATE] l2 (Lane)
  snowDepth: 0.0
  isFrozen: false
  gravelDepth: 0.0
  saltEffect: 0.0
  isBlocked: false
  vehicles: []
[STATE] l1 (Lane)
  snowDepth: 20.0
  isFrozen: false
  gravelDepth: 0.0
  saltEffect: 0.0
  isBlocked: false
  vehicles: []

Teszteset 10: Hókotró takarítása hóhányó fejjel

- **Leírás:** A gép hóhányó fejjel hajt rá egy hóval fedett sávra. Az áthaladás után a sáv lekérdezésekor a hómennyiség lenullázódik, de a szomszédos sáv paraméterei nem változnak meg (a hó a járdára kerül).
- **Ellenőrzött funkcionalitás, várható hibahelyek:** A ThrowerHead.clean() működésének verifikálása, amely véglegesen eltávolítja a havat a szomszédos utcák terhelése nélkül.
- **Bemenet:**

random off
create lane l_1
create lane l_2
connect_fields l_1 l_2
set_lane_state l_1 15.0 false 0.0 5.0
set_lane_state l_2 0.0 false 0.0 0.0
create homebase hb_1
connect_fields hb_1 l_1
create cleaner c_1
set_money c_1 2000
create snowplow sp_1
spawn snowplow sp_1 hb_1 c_1
buy c_1 sp_1 throwerhead
equip sp_1 throwerhead
move_plow sp_1 l_1
stat l_1
stat l_2

- **Elvárt kimenet:**

[SUCCESS] random mode set to off
[SUCCESS] lane l_1 created
[SUCCESS] lane l_2 created
[SUCCESS] fields l_1 and l_2 connected
[SUCCESS] lane l_1 state updated

[SUCCESS] lane l_2 state updated
[SUCCESS] homebase hb_1 created
[SUCCESS] fields hb_1 and l_1 connected
[SUCCESS] cleaner c_1 created
[SUCCESS] money set to 2000 for c_1
[SUCCESS] snowplow sp_1 created
[SUCCESS] sp_1 spawned at hb_1
[SUCCESS] c_1 bought throwerhead for sp_1
[SUCCESS] sp_1 equipped with throwerhead
[SUCCESS] sp_1 moved to l_1
[STATE] l_1 (Lane)
  snowDepth: 0.0
  isFrozen: false
  gravelDepth: 0.0
  saltEffect: 0.0
  isBlocked: false
  vehicles: []
[STATE] l_2 (Lane)
  snowDepth: 0.0
  isFrozen: false
  gravelDepth: 0.0
  saltEffect: 0.0
  isBlocked: false
  vehicles: []

Teszteset 11: Hókotró takarítása jégtörő fejjel

- **Leírás:** A jégtörő fejjel felszerelt gép fagyott sávra lép. A takarítás után a sáv nem lehet fagyott, és a jégvastagságnak nullára kell csökkennie.
- **Ellenőrzött funkcionalitás, várható hibahelyek:** Az IcebreakerHead.clean() jégzúzó logikáját ellenőrzi.
- **Bemenet:**

random off
create lane l1
create lane l2
connect_fields l1 l2
set_lane_state l2 0.0 true 4.0 0.0
create homebase hb1
connect_fields hb1 l1
create cleaner c1
create snowplow sp1
spawn snowplow sp1 hb1 c1
set_money c1 1000
buy c1 sp1 icebreakerhead
equip sp1 icebreakerhead
move_plow sp1 l1
move_plow sp1 l2
stat l2

- **Elvárt kimenet:**

[SUCCESS] random mode set to off
[SUCCESS] lane l1 created

[SUCCESS] lane l2 created
[SUCCESS] l1 connected to l2
[SUCCESS] lane l2 state updated
[SUCCESS] homebase hb1 created
[SUCCESS] hb1 connected to l1
[SUCCESS] cleaner c1 created
[SUCCESS] snowplow sp1 created
[SUCCESS] sp1 spawned at hb1
[SUCCESS] money set to 1000.0 for c1
[SUCCESS] c1 bought icebreakerhead for sp1
[SUCCESS] sp1 equipped with icebreakerhead
[SUCCESS] sp1 moved to l1
[SUCCESS] sp1 moved to l2
[STATE] l2 (Lane)
  snowDepth: 0.0
  isFrozen: false
  gravelDepth: 0.0
  saltEffect: 0.0
  isBlocked: false
  vehicles: []

Teszteset 12: Sóhatás kialakítása sószóró fejjel

- **Leírás:** A sószóróval felszerelt gép végighalad egy sávon. Az áthaladás után a sáv saltEffect értékének be kell állnia, a fej sókészletének pedig csökkennie kell.
- **Ellenőrzött funkcionalitás, várható hibahelyek:** A SaltHead.clean() és a saltEffect létrehozását, valamint az üzemanyag (só) fogyását ellenőrzi.
- **Bemenet:**

random off
create lane l1
create lane l2
connect_fields l1 l2
set_lane_state l2 0.0 true 2.0 0.0
create homebase hb1
connect_fields hb1 l1
create cleaner c1
create snowplow sp1
spawn snowplow sp1 hb1 c1
set_money c1 1000
buy c1 sp1 salthead
buy c1 sp1 salt 5
equip sp1 salthead
move_plow sp1 l1
move_plow sp1 l2
stat l2
stat sp1

- **Elvárt kimenet:**

[SUCCESS] random mode set to off
[SUCCESS] lane l1 created
[SUCCESS] lane l2 created
[SUCCESS] l1 connected to l2
[SUCCESS] lane l2 state updated

[SUCCESS] homebase hb1 created
[SUCCESS] hb1 connected to l1
[SUCCESS] cleaner c1 created
[SUCCESS] snowplow sp1 created
[SUCCESS] sp1 spawned at hb1
[SUCCESS] money set to 1000.0 for c1
[SUCCESS] c1 bought salthead for sp1
[SUCCESS] c1 bought 5.0 salt for sp1
[SUCCESS] sp1 equipped with salthead
[SUCCESS] sp1 moved to l1
[SUCCESS] sp1 moved to l2
[STATE] l2 (Lane)
  snowDepth: 0.0
  isFrozen: true
  gravelDepth: 0.0
  saltEffect: 3.0
  isBlocked: false
  vehicles: []
[STATE] sp1 (SnowPlow)
  currentField: l2
  activeHead: salthead
  fuelAmount: 4.0
  attachments: []

Teszteset 13: Zúzalék szórása az úttestre

- **Leírás:** A zúzalékszóróval felszerelt gép jeges sávra lép. A sáv gravelDepth értéke nő, a fej készlete csökken, miközben a jég továbbra is megmarad.
- **Ellenőrzött funkcionalitás, várható hibahelyek:** A GravelHead.clean() és a csúszásgátló zúzalékszórás működését ellenőrzi.
- **Bemenet:**

random off
create lane l1
create lane l2
connect_fields l1 l2
set_lane_state l2 0.0 true 3.0 0.0
create homebase hb1
connect_fields hb1 l1
create cleaner c1
create snowplow sp1
spawn snowplow sp1 hb1 c1
set_money c1 1000
buy c1 sp1 gravelhead
buy c1 sp1 gravel 5
equip sp1 gravelhead
move_plow sp1 l1
move_plow sp1 l2
stat l2

- **Elvárt kimenet:**

[SUCCESS] random mode set to off
[SUCCESS] lane l1 created
[SUCCESS] lane l2 created

[SUCCESS] l1 connected to l2
[SUCCESS] lane l2 state updated
[SUCCESS] homebase hb1 created
[SUCCESS] hb1 connected to l1
[SUCCESS] cleaner c1 created
[SUCCESS] snowplow sp1 created
[SUCCESS] sp1 spawned at hb1
[SUCCESS] money set to 1000.0 for c1
[SUCCESS] c1 bought gravelhead for sp1
[SUCCESS] c1 bought 5.0 gravel for sp1
[SUCCESS] sp1 equipped with gravelhead
[SUCCESS] sp1 moved to l1
[SUCCESS] sp1 moved to l2
[STATE] l2 (Lane)
  snowDepth: 0.0
  isFrozen: true
  gravelDepth: 1.0
  saltEffect: 0.0
  isBlocked: false
  vehicles: []

Teszteset 14: Komplex takarítás sárkány fejjel

- **Leírás:** A sárkány fejjel felszerelt gépet egyszerre havas és jeges sávra léptetjük. Egy lépés után minden téli paraméternek nullára kell csökkennie, a biokerozin készletnek pedig fogynia kell.
- **Ellenőrzött funkcionalitás, várható hibahelyek:** A DragonHead.clean() teljes olvasztó hatását ellenőrzi.
- **Bemenet:**

random off
create lane l1
create lane l2
connect_fields l1 l2
set_lane_state l2 6.0 true 5.0 0.0
create homebase hb1
connect_fields hb1 l1
create cleaner c1
create snowplow sp1
spawn snowplow sp1 hb1 c1
set_money c1 2000
buy c1 sp1 dragonhead
buy c1 sp1 kerosine 5
equip sp1 dragonhead
move_plow sp1 l1
move_plow sp1 l2
stat l2

- **Elvárt kimenet:**

[SUCCESS] random mode set to off
[SUCCESS] lane l1 created
[SUCCESS] lane l2 created
[SUCCESS] l1 connected to l2
[SUCCESS] lane l2 state updated

[SUCCESS] homebase hb1 created
[SUCCESS] hb1 connected to l1
[SUCCESS] cleaner c1 created
[SUCCESS] snowplow sp1 created
[SUCCESS] sp1 spawned at hb1
[SUCCESS] money set to 2000.0 for c1
[SUCCESS] c1 bought dragonhead for sp1
[SUCCESS] c1 bought 5.0 kerosine for sp1
[SUCCESS] sp1 equipped with dragonhead
[SUCCESS] sp1 moved to l1
[SUCCESS] sp1 moved to l2
[STATE] l2 (Lane)
  snowDepth: 0.0
  isFrozen: false
  gravelDepth: 0.0
  saltEffect: 0.0
  isBlocked: false
  vehicles: []

Teszteset 15: Takarítási kísérlet kifogyott üzemanyaggal

- **Leírás:** 0 készlettel (üzemanyag nélkül) rendelkező sószóróval hajtunk fagyott sávra. A mozgás sikeres, de a sáv állapota nem változhat meg, mivel nincs anyag a szóráshoz.
- **Ellenőrzött funkcionalitás, várható hibahelyek:** Az Empty state és a feltételes clean-végrehajtás működését ellenőrzi.
- **Bemenet:**

random off
create lane l1
create lane l2
connect_fields l1 l2
set_lane_state l2 0.0 true 2.0 0.0
create homebase hb1
connect_fields hb1 l1
create cleaner c1
create snowplow sp1
spawn snowplow sp1 hb1 c1
set_money c1 1000
buy c1 sp1 salthead
equip sp1 salthead
move_plow sp1 l1
move_plow sp1 l2
stat l2

- **Elvárt kimenet:**

[SUCCESS] random mode set to off
[SUCCESS] lane l1 created
[SUCCESS] lane l2 created
[SUCCESS] l1 connected to l2
[SUCCESS] lane l2 state updated
[SUCCESS] homebase hb1 created
[SUCCESS] hb1 connected to l1
[SUCCESS] cleaner c1 created

[SUCCESS] snowplow sp1 created
[SUCCESS] sp1 spawned at hb1
[SUCCESS] money set to 1000.0 for c1
[SUCCESS] c1 bought salthead for sp1
[SUCCESS] sp1 equipped with salthead
[SUCCESS] sp1 moved to l1
[SUCCESS] sp1 moved to l2
[STATE] l2 (Lane)
  snowDepth: 0.0
  isFrozen: true
  gravelDepth: 0.0
  saltEffect: 0.0
  isBlocked: false
  vehicles: []

Teszteset 16: Körök múlása és havazás szimulációja

- **Leírás:** Több next_turn parancsot adunk ki, hogy kiváltsuk a globális havazást. A lane objektumok hóvastagságának egyszerre kell növekednie.
- **Ellenőrzött funkcionalitás, várható hibahelyek:** A körléptetéshez kötött Map.snow() eseményt ellenőrzi.
- **Bemenet:**

random off
create lane l1
create lane l2
connect_fields l1 l2
set_lane_state l1 0.0 false 0.0 0.0
set_lane_state l2 0.0 false 0.0 0.0
next_turn
next_turn
next_turn
stat l1

- **Elvárt kimenet:**

[SUCCESS] random mode set to off
[SUCCESS] lane l1 created
[SUCCESS] lane l2 created
[SUCCESS] l1 connected to l2
[SUCCESS] lane l1 state updated
[SUCCESS] lane l2 state updated
[EVENT] Map snowed. Global depth increased.
[EVENT] Map snowed. Global depth increased.
[EVENT] Map snowed. Global depth increased.
[STATE] l1 (Lane)
  snowDepth: 3.0
  isFrozen: false
  gravelDepth: 0.0
  saltEffect: 0.0
  isBlocked: false
  vehicles: []

Teszteset 17: Sóhatás idejének lejárta

- **Leírás:** Előre sózott sávot többször léptetünk a körökkel. A saltEffect értékének fokozatosan nullára kell csökkennie.

- **Ellenőrzött funkcionalitás, várható hibahelyek:** A sávok időzített állapotfrissítését ellenőrzi.
- **Bemenet:**

random off
create lane l1
set_lane_state l1 0.0 false 0.0 0.0
set_salt_effect l1 2.0
next_turn
next_turn
stat l1

- **Elvárt kimenet:**

[SUCCESS] random mode set to off
[SUCCESS] lane l1 created
[SUCCESS] lane l1 state updated
[SUCCESS] salt effect set to 2.0 for l1
[EVENT] Turn advanced
[EVENT] Turn advanced
[STATE] l1 (Lane)
  snowDepth: 0.0
  isFrozen: false
  gravelDepth: 0.0
  saltEffect: 0.0
  isBlocked: false
  vehicles: []

Teszteset 18: Felszerelés vásárlása a telephelyen

- **Leírás:** A takarító egy telephelyen álló hókotróhoz új sószóró fejet vásárol. A tranzakció után a játékos pénze csökken, az új fej pedig bekerül a gép attachments listájába.
- **Ellenőrzött funkcionalitás, várható hibahelyek:** A market sikeres buy ágát és a pénzlevonás konzisztenciáját ellenőrzi.
- **Bemenet:**

random off
create homebase hb1
create cleaner c1
create snowplow sp1
spawn snowplow sp1 hb1 c1
set_money c1 100
buy c1 sp1 salthead
stat c1

- **Elvárt kimenet:**

[SUCCESS] random mode set to off
[SUCCESS] homebase hb1 created
[SUCCESS] cleaner c1 created
[SUCCESS] snowplow sp1 created
[SUCCESS] sp1 spawned at hb1
[SUCCESS] money set to 100.0 for c1
[SUCCESS] c1 bought salthead for sp1
[STATE] c1 (Cleaner)
  name: c1
  money: 50.0

score: 0

Teszteset 19: Vásárlási kísérlet fedezet nélkül

- **Leírás:** A játékos a pénzénél drágább felszerelést próbál vásárolni. A rendszernek hibát kell jeleznie, a pénzegyenleg nem mehet mínuszba, és új attachment nem jelenhet meg.
- **Ellenőrzött funkcionalitás, várható hibahelyek:** Az elégtelen fedezet kezelését és az állapotintegritást ellenőrzi.
- **Bemenet:**

```
random off
create homebase hb1
create cleaner c1
create snowplow sp1
spawn snowplow sp1 hb1 c1
set_money c1 10
buy c1 sp1 dragonhead
stat c1
```

- **Elvárt kimenet:**

```
[SUCCESS] random mode set to off
[SUCCESS] homebase hb1 created
[SUCCESS] cleaner c1 created
[SUCCESS] snowplow sp1 created
[SUCCESS] sp1 spawned at hb1
[SUCCESS] money set to 10.0 for c1
[ERROR] c1 has insufficient funds for dragonhead
[STATE] c1 (Cleaner)
  name: c1
  money: 10.0
  score: 0
```

Teszteset 20: Felszereléscsere megkísérlése az utcán

- **Leírás:** A hókotró két fejjel rendelkezik, de nem telephelyen áll. Az equip parancsnak hibát kell adnia, az aktív fej pedig változatlan kell maradjon.
- **Ellenőrzött funkcionalitás, várható hibahelyek:** A helyszín-specifikus akciók kikényszerítését ellenőrzi.
- **Bemenet:**

```
random off
create lane l1
create homebase hb1
connect_fields hb1 l1
create cleaner c1
create snowplow sp1
spawn snowplow sp1 hb1 c1
set_money c1 1000
buy c1 sp1 sweephead
buy c1 sp1 salthead
equip sp1 sweephead
move_plow sp1 l1
equip sp1 salthead
```

- **Elvárt kimenet:**

```
[SUCCESS] random mode set to off
[SUCCESS] lane l1 created
```

[SUCCESS] homebase hb1 created
[SUCCESS] hb1 connected to l1
[SUCCESS] cleaner c1 created
[SUCCESS] snowplow sp1 created
[SUCCESS] sp1 spawned at hb1
[SUCCESS] money set to 1000.0 for c1
[SUCCESS] c1 bought sweephead for sp1
[SUCCESS] c1 bought salthead for sp1
[SUCCESS] sp1 equipped with sweephead
[SUCCESS] sp1 moved to l1
[ERROR] sp1 cannot change head outside homebase

9.2 A tesztelést támogató programok tervei

1. Egy teszt lefuttatása hogyan fog zajlani?

- Egyetlen teszteset végrehajtása operációs rendszer szintű Standard I/O (bemenet/kimenet) átirányítással történik.
- A felhasználó vagy a teszt-szkript elindítja a lefordított Java alkalmazást a parancssorból.
- A billentyűzetről történő gépelés helyett a < operátorral egy előre megírt bemeneti szövegfájlt (pl. test01_in.txt) adunk át a programnak.
- A konzolra történő kiírást a > operátorral egy kimeneti szövegfájlba (pl. test01_out.txt) irányítjuk át.
- **Futtatási parancs:** java ProtoApp < test01_in.txt > test01_out.txt
- A teszt lefutása után a program terminál, a kimeneti fájl pedig tartalmazza a memóriában lezajlott folyamatok eredményét a kimeneti nyelv formátumában.

2. Tesztek lefuttatása támogatással (RunAllTests.bat terve)

- A tömeges teszteléshez egy Windows Batch fájlt (RunAllTests.bat) készítünk, amely a projekt gyökérmappájából indítva teljesen automatizálja a 20 teszteset végrehajtását.
- **A RunAllTests.bat algoritmusa (pszeudokód):**

1. Beállítja a megfelelő kódlapot az ékezetes karakterek és a tiszta konzolos megjelenítés érdekében.
2. Lefordítja a Java forráskódot a javac paranccsal (ha szükséges).
3. CIKLUS MINDEN *_in.txt kiterjesztésű fájlra a tesztek mappájában: 3.1. Kinyeri az aktuális fájl alapnevét (pl. test01_in.txt -> test01). 3.2. Futtatja a prototípust I/O átirányítással: bemenetként megadja az aktuális *_in.txt fájlt, kimenetként létrehozza a *_out.txt fájlt. 3.3. Meghívja a kimenet-összehasonlító rutint (lásd 3. pont) az elkészült fájlra.
4. A tesztelés végén összesíti, és kiírja a konzolra a sikeres és sikertelen tesztek számát.

3. Elvárt és tényleges kimenet összehasonlítása hogyan zajlik?

- Az összehasonlítást a Windows beépített parancssori segédprogramja, az fc (File Compare) végzi. A RunAllTests.bat szkript a generált kimenetet karakterre pontosan összeveti a fejlesztők által előre megírt, elvárt kimenettel.
- **Az összehasonlító rutin algoritmusa (a cikluson belül):**

1. Futtatja a fájl-összehasonlító parancsot némítva: fc alapnév_out.txt alapnév_expected.txt > nul
2. Lekérdezi az operációs rendszertől a parancs visszatérési értékét.
3. HA a visszatérési érték 0 (a két fájl bájtra pontosan megegyezik), AKKOR: 3.1. Kiírja a konzolra: [PASS] - alapnév 3.2. Megnöveli a sikeres tesztek számlálóját.

4. KÜLÖNBEN (ha a visszatérési érték > 0, azaz eltérés van): 4.1. Kiírja a konzolra: [FAIL] - alapnév 4.2. (Opcionális) Futtatja újra az fc parancsot, de az eredményét kimenti egy alapnév_diff.txt fájlba, hogy a fejlesztő lássa, pontosan melyik sorban és karakterben tért el a generált kimenet az elvárttól. 4.3. Megnöveli a sikertelen tesztek számlálóját.

9.3 Napló

Kezdet	Időtartam	Résztvevők	Leírás
2026.03.19 19:00	4 óra	Kálmán Héja Papp Grób Dobronyi	Értekezlet. Döntés: Megállapodtunk hogy ki mit készít el, és hogy hogyan integráljuk a változtatásokat.
2026.03.20 20:00	3 óra	Kálmán	Tevékenység: változások integrálását.
2026.03.21. 18:00	3 óra	Héja	Tevékenység: Elkészítette az osztályok tervének a felét.
2026.03.21. 02:00	3 óra	Papp	Tevékenység: Elkészítette az osztályok tervének a másik felét.
2026.03.21. 10:00	3 óra	Grób	Tevékenység: Elkészítette a tesztek felét
2026.03.22. 14:00	3 óra	Dobronyi	Tevékenység: Elkészítette a tesztek másik felét.
2026.03.22. 18:00	4 óra	Kálmán Héja Papp Grób Dobronyi	Értekezlet. Döntés: összeegyeztettük az egyénileg megcsinált részfeladatokat, és véglegesítettük a dokumentumot.